Individual Project
Final Report

ID Number: B930988

Programme: Product Design Engineering

Module Code: 21WSC501

Project Title: Feasibility of Integrating Kinect V2 for VR Sports Simulation

1 Abstract

The main aim of the project is to integrate lower body tracking using affordable solutions to Unity VR. Compared to the initial concept of the project, it has been narrowed down to compensate for my lack of knowledge in game development and coding. However, the integration of Kinect motion capture was successful and made a meaningful project. The creation of skateboard tutorial mechanics and physics were unachievable within the time scope given, however, the feasibility of such a product was shown within the timeframe.

2 Contents

1 Abstract 1
3 Introduction 3
4 Literature Review 4
4.1 Use of VR technology in Sports 4
5 Learning to use Unity 6
5.1 Exploring with Kinect Studio 6
5.2 Tracking capability 6
5.3 Utilising asset from Unity Asset store 6
6 Method 7
7 UI design for menu 10
8 Learning to code for Unity 12
8.1 Creating Hand model 12
9 Animation rigging and Inverse Kinematics (IK) 14
10 Conclusion 17
10.1 Future improvements and implementation 17
10.2 what can this be used for? 17
11 Appendices 19
12 References 28

3 Introduction

The project aims to develop an affordable solution to upper and lower body tracking in virtual reality (VR) in the form of a skateboard tutorial setup. As of current, VR body tracking solutions are limited and often expensive for average consumers and developers. The project focuses on the use of Kinect V2 sensors to capture the body movement and the implementation of it to skateboarding.

VR has become the forefront of many fields of technology such as athlete training, medical treatment (rehabilitations), automobile, manufacturing, entertainment, education etc. The project aims to integrate Kinect motion capture in VR to control the body movement to help skateboard beginners learn the absolute basics of skateboarding.

To perform a skateboard trick in real life, it requires precise foot motion and repeated muscle memory training. Beginners may find it difficult committing to entry tricks such as Ollie simply because it requires time and effort. Also, there is a very high possibility for beginners to get injured which could demotivate them from learning further. The integration of motion capture can help create a simulation that support beginners learn the basics and give interest to people who has never skated before.

The main objective for this project will be to research the feasibility of implementing Kinect motion capture to unity and evaluation on whether it is suitable for the VR skateboard tutorial.

The report contains the process of VR game development and motion capture method using Kinect V2.

It is important to note that the research for this topic was difficult to find online as it is a novel topic, and yet to have published meaningful research and results using Kinect V2 as a motion capture system in VR. Most of the resources were obsolete, published more than 6 years ago which does not specifically apply to my topic. Recently Unity and Microsoft Kinect have made various changes to their software and functions which made the development optimised for VR however, it can be more complicated as new resources are limited.

Only couple of commercial VR skateboard game exists, VR Skater and Hover Skate VR however, the resources are not open to public and it is far from realistic skateboarding experience. The game utilises VR controller as main control system in game, using buttons and joysticks to perform tricks and rowing motion to control pushing and turning. The game takes you to fully immersive skating environments, but it is purely for entertainment purposes and does not provide a training environment for real life skating.

These games have potential to be developed further for real life training if motion capture is integrated to control skateboard using live data of your body movement. However, a commercial motion tracker can cost as much as £130 each and considering you require at least three trackers for lower body, it is not a viable option for many consumers. This was the initial motivation for the project to develop an affordable motion capture system that can be integrated in VR for sports training purposes.

4 Literature Review

A study conducted by Benjamin.H.Groh provides useful information on the visualisation of tricks using an accelerometer and gyro sensors and whether it is useful for the skate community. The result of the study suggests that 27 experienced skaters who were involved in the survey found that it was useful to visualise their tricks for their training purposes. [2] The survey was done in seven-point Likert scale, 1 indicating “strongly disagree” and 7 indicating “strongly agree”. The median of results fell between 5 and 6 which indicated the importance of visualisation of tricks. Participants suggested possible improvements in visualisation of foot and body movements, establishment of training application and multiple perspectives.

In terms of validity of using Kinect V2 as motion capture system, the study was conducted to evaluate the accuracy of movement tasks captured using two Kinect V2s adjacent to each other. The study concluded that it tracks each movement task with high levels of consistency, specifically, tracking joint displacements. However, some body parts were recorded with less accuracy particularly foot and ankle displacement.

Much research has been carried out for the application of Kinect V2 for upper limb tracking and its accuracy for various uses (i.e., gaming, medical rehabilitation, human computer interaction). However, this study suggests that increasing accuracy of Kinect V2 requires calibration and extra sensor(s) to be acceptable of use in professional environments.

For this project, only one Kinect V2 sensor is used for motion capturing which may further affect the accuracy of tracking lower body movements. However, with constraints in budget and skillset the project is based on using one Kinect V2 sensor and work around it to get the best outcome.

4.1 Use of VR technology in Sports

Athlete participation in sports related simulation in VR has many applications and benefits. Some professional sports organisations such as NBA, FIFA and NFL have already tested and proved its viability in competitive sports. In sports training, repetition is a very important aspect to enhance a player’s ability. [3] However, in real life it can be difficult to manage. Various factors can affect players from repetitive training such as weather condition, body fatigue, availability and lack of entertainment. STRIVR is one of the forefront engineers to develop VR training for athletes. In one of their case studies, an experienced NBA player tackled his problem in low percentage free throw makes using their program to train for his free throw accuracy. [12] In his career, he averaged 59.7% in free throw shots, considered low in the NBA (2016-2017 NBA League average 77.2%). [13] He trained with STRIVR for three weeks in 2016-2017 season, which he improved his percentage by approximately 30%, compared to the percentage of non-STRIVR training weeks. This significant difference shows that VR training for sports is effective to enhancing a player’s performance.

Dan Li conducted a survey to find out whether VR sports training attracts interest from college athletes. 30 college students participated in the survey were asked whether VR sports training spark interests. [14] 67% showed interest in training with VR. Their reasoning being that it was

more exciting and would improve training efficiency. Also, training can be conducted freely, without having to worry about other people watching them, which proved to increase player's self-confidence.

In skateboarding, especially for beginners, it might be intimidating to go out and practice. Beginners feel self-conscious about how they would look to other people. In VR training they can build up the confidence to achieve the basics and increase confidence and consistency on landing tricks in real life.

Based on the research, the project originally aimed to build a VR simulation to tackle this problem however, with underestimation of the task and having to learn to program with C#, Unity, and Microsoft Kinect integration it was not possible to achieve.

Project process was taken step by step to set up the basis to develop VR simulation using Kinect V2 and resources that could be integrated in the game.

5 Learning to use Unity

With no previous experience, the dedicating the time to learn C# was required. Learning resources provided by Unity first to make a simple non-VR 3D game. It took some time to get familiar with using the software effectively especially for VR. Coding C# was very challenging as there are many components of coding when making a game such as physics, UI, motion capture and animation. The coding did not include the physics required in the game; however, all the resources and analysis are present to build the simulation in the future.

5.1 Exploring with Kinect Studio

In order to get the Kinect working on windows, installation of Kinect for Windows SDK 2.0 (developer tool) from Microsoft is required. [6] Also, to use Kinect in Unity, the extended package Unity Pro is required to output the Kinect data into Unity.

There are various things can be done just by simply installing this software. Kinect can show live camera output to unity and output skeletons. However, to assign skeleton's joint coordinates to unity character model, complex coding is required.

5.2 Tracking capability

Kinect V2 tracks 25 skeletal (5 new joints were added to the new one). The skeletal joints are shown below in figure 1.

These accurate joint positions can be aligned with character models in Unity to work in correspondence to live body movement data. However, as Kinect does not detect fingers for each hand, it requires separate construction of the hand model in Unity.

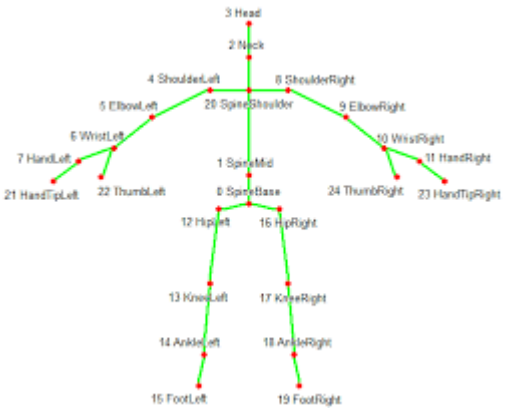

Figure 1 Joints allocation on Kinect

5.3 Utilising assets from the Unity Asset store

To test the accuracy and feasibility of Kinect V2 sensor for motion capture, there is an asset made by RF solutions which include various Demo scenes that I used to analyse the accuracy.

Demos that could be useful for skateboard tutorial (such as Motion Capture recording, Hand detection, Heigh detection and Pose detection) were chosen.

6 Method

First, importing Kinect SDK 2.0 and Unity pro package file from the Microsoft developer website was required to output the data into Unity. [6] The process was simple as only required to drop the files to project in Unity and when scene is played, Kinect V2 behaves as a webcam.

To get the character respond real time in conjunction with Kinect, an asset from Unity asset store was imported to reduce the time of coding complexion. Importing this asset was somewhat complex however, it drastically lessened the effort of coding for this section. Using this asset, valuable data and ideas about how the character reacts in Unity was obtained. There are four possible useful components discovered. Hand detection, Height detection, motion capture recording and pose detection. Hand and height detection can be used for overall coordinate data in Unity to how accurately real time data is received to the character. Figure_ shows the hand is only detected when the palm is faced towards and put in front of the body. This is due to the lack of spatial recognition of Kinect V2, it is only capable of detecting mediolateral direction. [8] This is a problem as whenever the hand is out of position, the character's hand will not detect the movement from Kinect. This then creates the need to create a separate hand model with physics involved in Unity. More on that later.

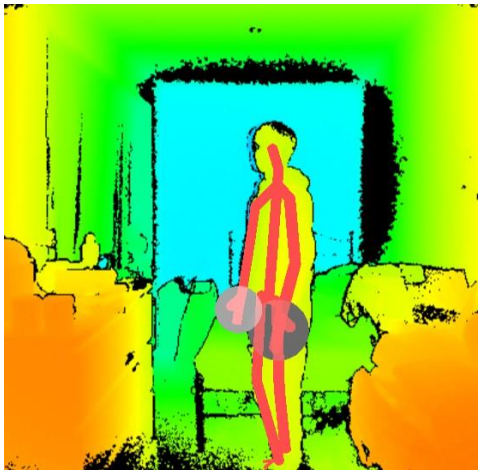

Figure 2 Kinect not recognising >60º body turn

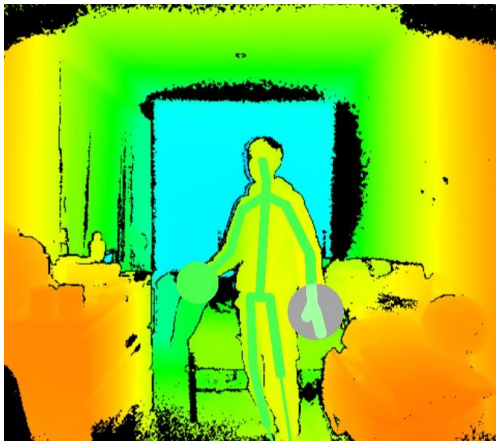

Figure 3 Kinect detects body at 45º > body turn

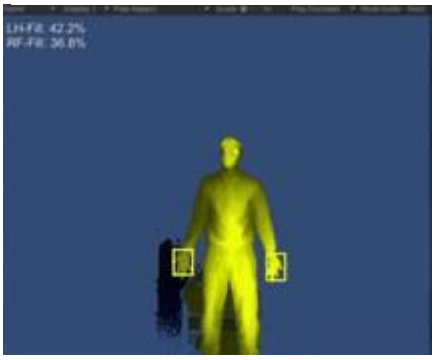

Figure 4 hand detection palm facing backwards to camera

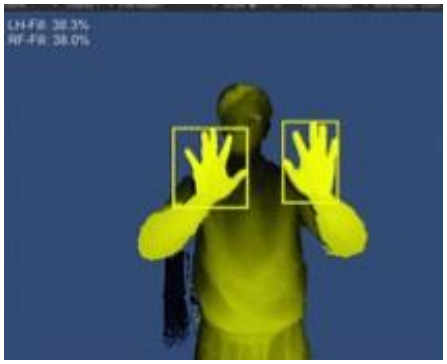

Figure 5 Kinect not detecting perfectly close hands palm facing forwards

Figure 6 only one hand detected when faced body sideways

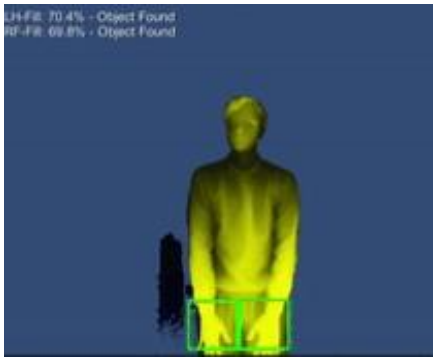

Figure 7 Kinect detecting hands perfectly palm facing towards the body

Height detection (figure_) shows the accuracy of the body captured and proportioned in Unity. This data can be used to adjust the main camera height in VR according to the user's height and will give the appropriate head position. The motion capture recording can be used simply to animate the character in Unity and can be implemented to the skateboard simulation to show the player what movement task they need to carry out before performing a trick.

The motion capture recording did not work appropriately, uncovering the weakness of Kinect sensor, the inaccuracy of lower body movement capture. In figure_, the real time model's (on the left) leg is glitched during the jump motion recording. There is research conducted by University of Bournemouth to smooth out the animation and cleaning of character mesh, achieved by sending real time data as ASCII to a central server implemented in Raspberry Pi which then Unity pulls the data, and then outputs the corrected and smoothed out real time skeleton model. In this study, a quadricep stretch was performed which the Kinect raw data could not detect, and they have smoothed the animation using Kalman filter which then was corrected and properly detecting the quadricep stretch. [11] However, not enough material was presented in the study and was not feasible to implement into this project.

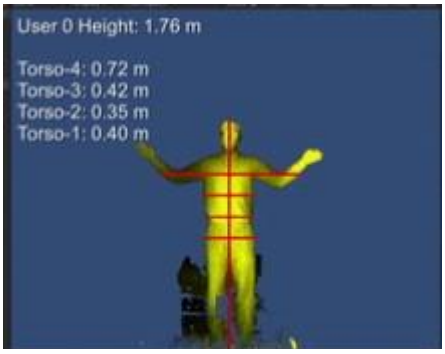

Figure 8 Height detection: Actual model's height matches perfectly with Kinect

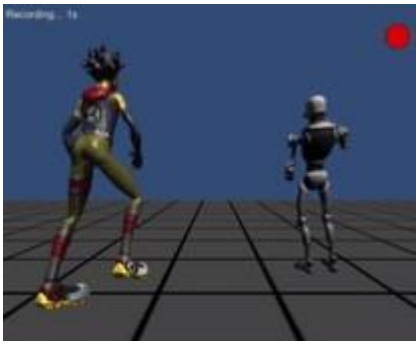

Figure 9 Height detection: Actual model's height matches perfectly with Kinect

Pose detection is the most useful resource that could be used in the simulation. It can be used as the feedback of trick performance and how close the player matched the trick in game. This demo conducted again shows the inaccuracy of Kinect V2 sensor as the highest pose matching value obtained was 85%. Although this number was achieved by not copying the exact pose but achieved by changing the orientation of the hands slightly. Going back to the hand detection, as it only recognises it at a certain position of hands the assumption is that the same problem occurred during the pose detection.

Figure 10 Motion capture recording

Figure 11 Motion capture recording result 1

These discoveries raise the question whether Kinect V2 is feasible for motion capturing in Unity, however it is still an accurate device for its cost and software availability.

Figure 13 Pose matching best score: 85%

Figure 12 Motion capture recording 2

7 UI design for menu

For the creation of simulation, UI design was an inevitable part of the development. The menu should be simple and intuitive to navigate the user into a quick setup which then leads into them diving into an immersive experience. The design was done by creating the draft in PowerPoint and then implementing the design to Unity. Figure 14-17 shows the wireframes for the menu system.

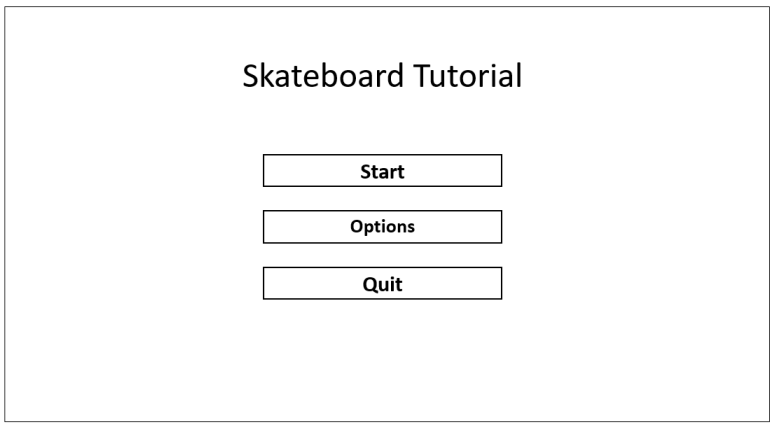

Figure 14 Main menu design

The menu is presented in front of the player clearly indicating the functions of each button. Coding was needed to make the menu system work as imagined. By clicking the Start button, it gets rid of the menu canvas and starts the scene in game. The option button refreshes the menu, taking the user to Options menu where the user can pick their stance on the skateboard, learn skate terms or go back to the main menu.

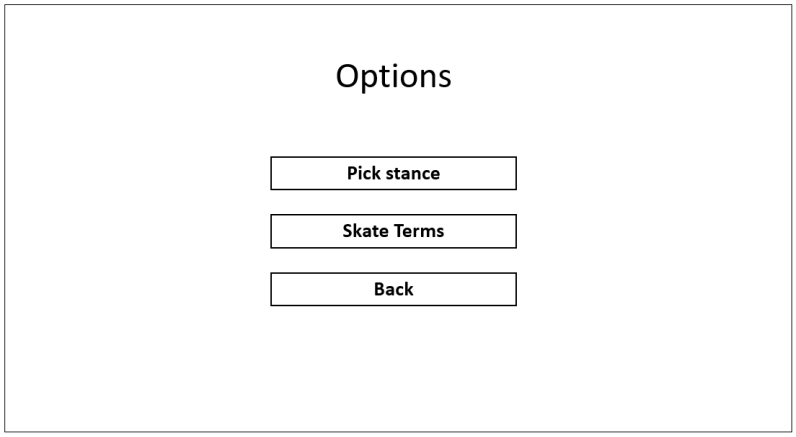

Figure 15 Options menu

By selecting “Pick stance”, the Stance menu is presented where the player can choose their preferred default stance. It also describes the meaning of each stance as the game is meant for beginners.

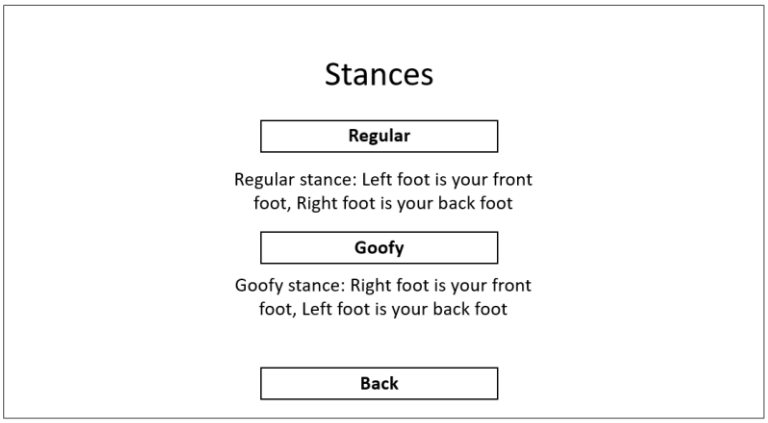

Figure 16 Stances menu

The "Skate terms" button takes the player to a menu displaying basic skate terms that they can learn to get familiar with skateboarding and by selecting back takes to the main menu.

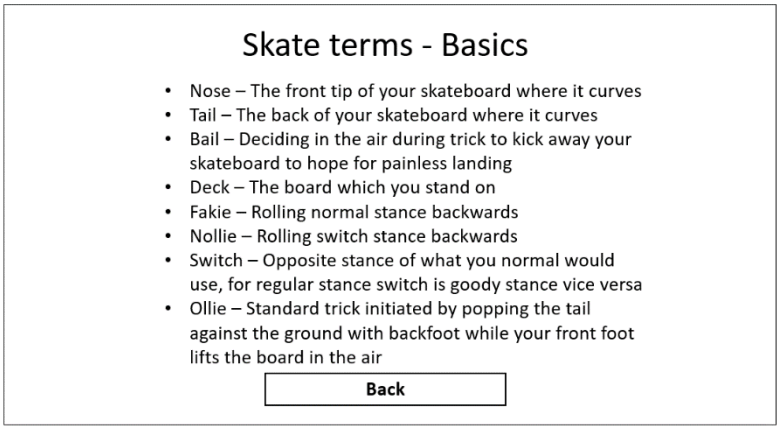

Figure 17 Skate terms menu

8 Learning to code for Unity

8.1 Creating Hand model

As explained in the previous section, there was a need to make a separate hand model with physics in VR. The coding for this was challenging as adding physics was a first experience. Unity's coding manual especially for the newly added package, XR interaction tool kit, was utilised. [10] As a learning opportunity, instead of using pre-existing asset package, this was all created from nothing to add animation and physics to the hand model.

So that animations and physics could be added to the model, layers of coding were required. The controller script gives the option to choose which game object is selected for the code to recognise. (See appendix. Figure 1 and 3)

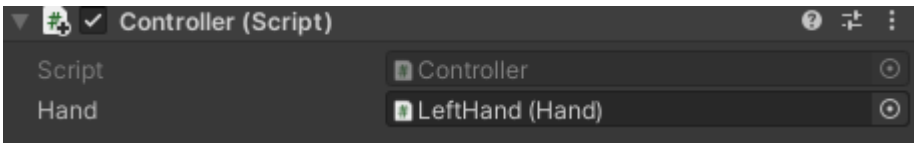

Figure 18 Controller script

The component ActionBasedController is called on script and the values grip and trigger both have a float value. The animator method is set to the Hand script which defines the movement, animation, and physics of each finger when the grip and trigger are performed. The moves are performed by pressing down the buttons on the VR controller and when grip is performed, the middle, ring and pinkie finger animate as grip, and the thumb and index finger as trigger. Then the animator window in Unity can execute the animation by configuring and recording the hand model for when the move is executed.

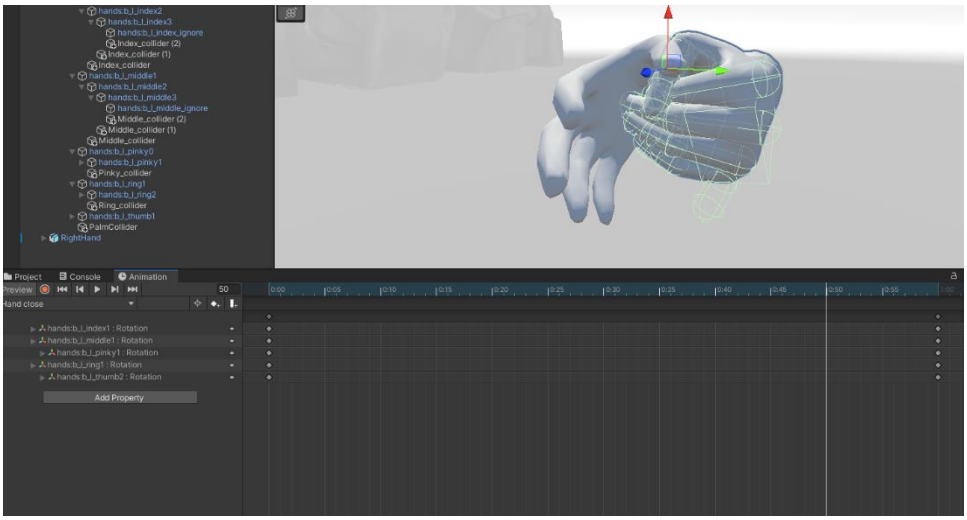

Figure 19 Hand animation recording

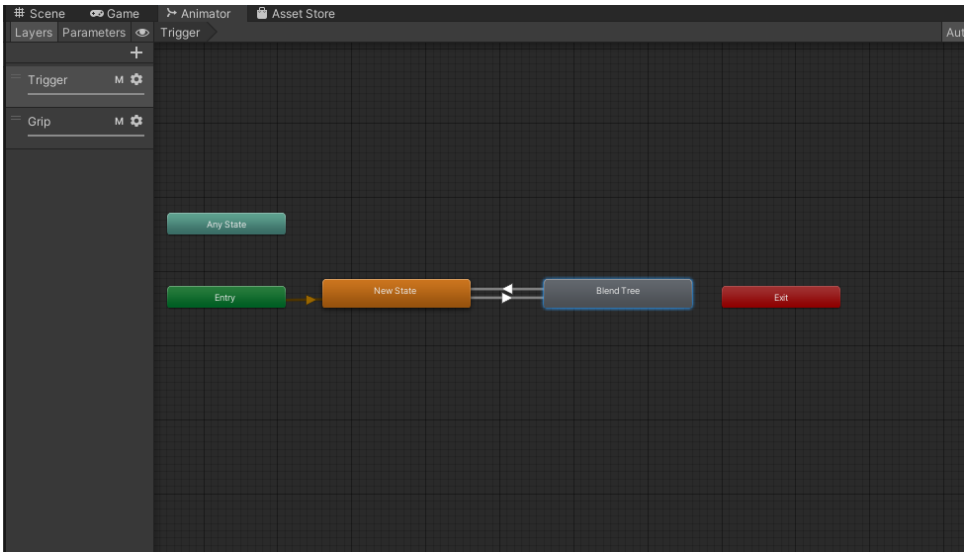

Figure 20 Animator window

When adding physics to any character and interactable object is complicated. On the same Hand script, collision detection and interpolation of the model need to be defined. This prevents the model from going through any game objects. Another function exists where when the game starts, the position of the hand model automatically teleports to where the controllers are held. This is done by simple mathematics in code, making a variable between the target position and transform of the hands and adding vectors. Adding rotation was more complex and required more research. A variable was set using quaternion which controls the rotation behaviour relating to each axis however when the written code was tested, the hands kept rotating in a random manner with a random velocity which it was not able to execute correctly. This was fixed by adding angular velocity to the hand and converting the quaternion value to float angle and vector3 axis. (See appendix. Figure 10)

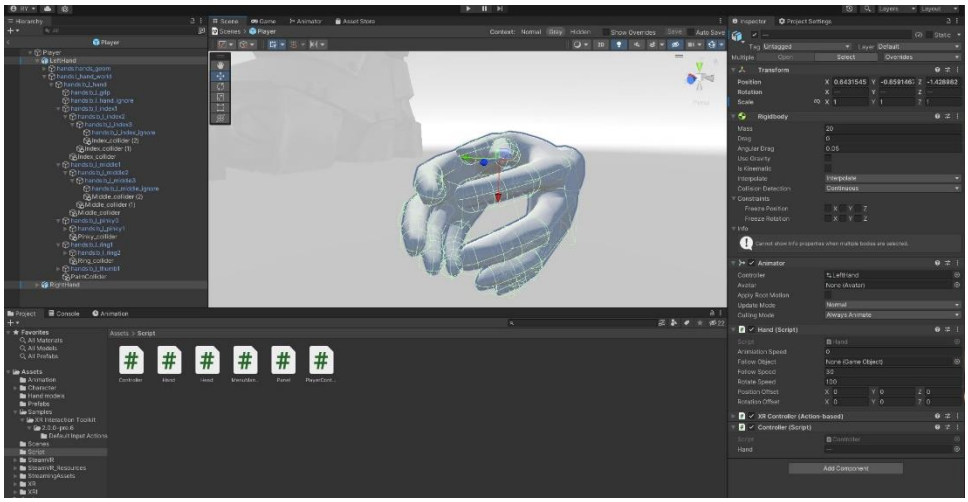

Figure 21 Adding physics to hand

9 Animation rigging and Inverse Kinematics (IK)

Inverse kinematics adds precise motion of each joint of skeleton in the character model.

By adding inverse kinematics to the body, it enhances the immersive experience in game. Without inverse kinematics, characters limbs are rigid and will not respond to the bending motion performed in real life. This section describes the method integrated to add inverse kinematics to the body of a character.

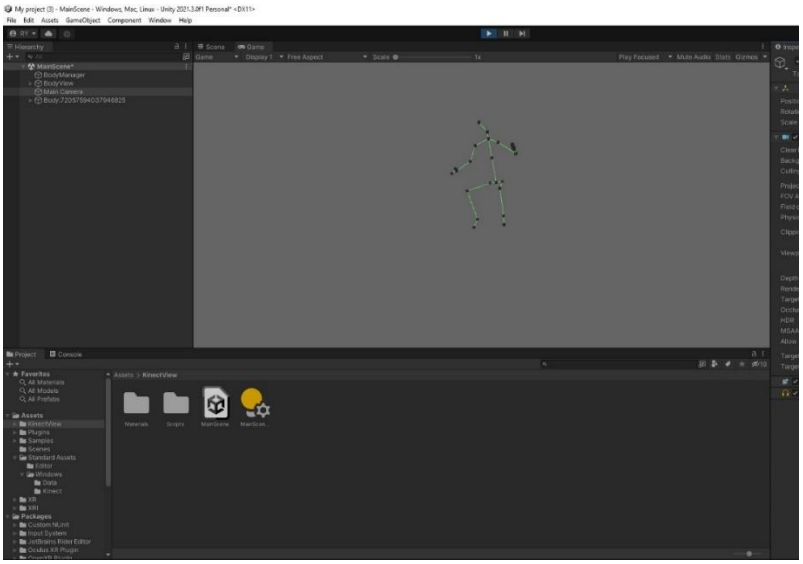

Figure 22 Kinect skeleton view on Unity

In Unity, a character to set up animation rigging is added. It allows the character to modify joint positions and rotations which is then used to integrate IK to each joint, also animation recordings of a character can be done by animation rigging. An empty game object for each joint was created. IK was added to the object, which then was assigned to the constraints of the skeleton of the character. In the empty object, the “Two bone IK constraints” component sets the reference to the weight and source object of the IK mechanism. Set up for the head was slightly different, instead of “Two bone IK constraints”, “Multi parent constraints” was added and then it was necessary to add the target to source object. Hint did not exist for the head as there was no joint direction reference needed.

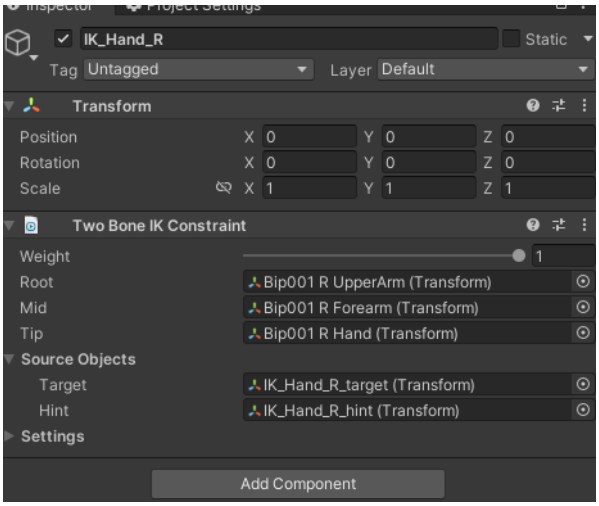

Figure 23 Inverse Kinematics rig set up

The transform of the character was aligned to the target to create the reference for the joint selected. The reference to hint tells Unity where the joint should be pointing. The same procedures were carried out for all joints of the body parts.

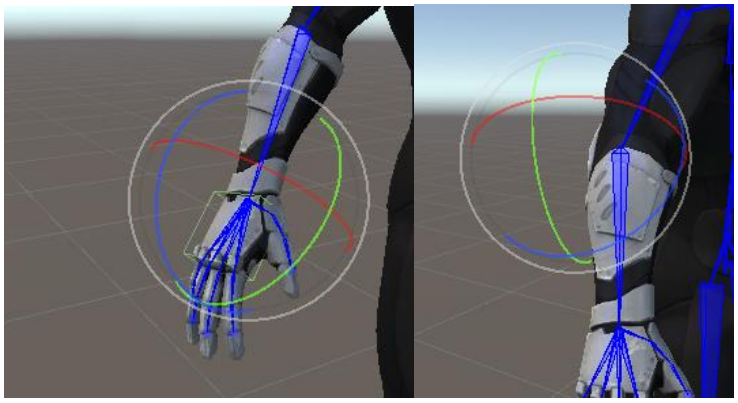

Figure 24 Inverse kinematics Target (left)Inverse Kinematics Hint (right

Some issues were encountered when testing at this point, it was solved by disabling the XR interaction manager in the project hierarchy.

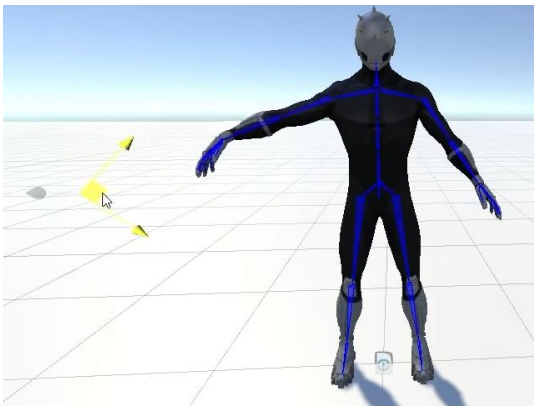

Figure 25 Testing the IK rig, successful as the arm does not stretch beyond the point and it points in the direction of stretch

Combining the IK body part with the actual model is where it fully demonstrates the IK model. A physicalized body part/joint is combined with the character model and the visualised body part (mesh) follows the target. Also, the target follows the controller and enabled by physics. To do so, the hand script created previously was added to each hand and modify the code to suite IK.

To add physics to the body, new scripts for head and legs would be added along with some modifications in the hand script. The head script adds physics so that the head follows the rotation and position of the head mounted display. (See appendix figure 2)

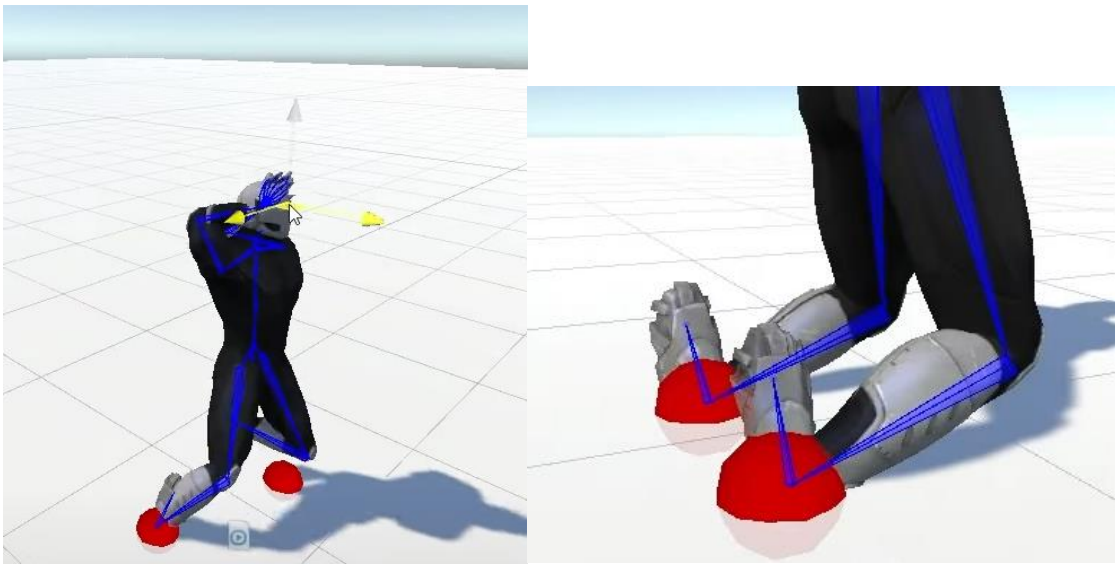

Figure 26 Issues with adding IK to lower body, twisted feet

There was an issue with foot positioning and orientation from the leg code. To solve this, the initial thought process was to add offsets to the foot position and rotation however, it only added more bugs to the system. The issue was not able to be fixed on time as the lack of coding skill made the search for a solution exceptionally long.

10 Conclusion

10.1 Future improvements and implementation

Although the project outcome may not satisfy the initial designs and ideas, there was a valuable gain in individual skill from it. Creating a functional VR simulation in conjunction with Kinect motion sensor and learning C# was underestimated during the initial design process. Having Kinect as a motion sensor is a feasible option however, if the Kinect was used to control a player, there would have to be complex coding for the IK of the model and its position and angle. There are not many solutions and products online which would make this project hard but motivational to get it working. Unfortunately, within the given time frame it could not achieve the satisfying point of development due to limitations in skills and encountering various errors that could not be solved swiftly.

Although, there were good findings in this project. When the Kinect was integrated into Unity, it demonstrated that Kinect V2 sensor was not accurate, particularly in motion capture recording. The upper body movement is captured with some decent accuracy however, due to the nature of the Kinect, it would not be able to capture movements when the body is angled and not facing the camera directly. It also struggled to capture the feet correctly, resulting in crooked legs in the motion capture recording. Combining that with IK would theoretically work however, there is a need to code the positioning and rotation of the feet from scratch. This alone can be a long process as it requires a number of iterations and testing with VR rig to get everything correctly.

The problem regarding the accuracy of the Kinect sensor can be solved by investing in higher-end motion sensors such as Azure Kinect or Xsens (marker-based), however, this solution requires a large facility with experienced technicians to utilise software which would then need to be converted to Unity format and it is far from an affordable solution. Another somewhat affordable solution can be having multiple Kinect sensors to accommodate full body capturing, setting cameras in the front, side and back of the body. [7] By adding multiple hardware increase the complexity of coding and error solving which may not be feasible for a lesser experienced developer.

10.2 what can this be used for?

From the method used, this project can be used to produce IK-based sports simulations using Kinect. It may not be suitable for sports such as skateboarding where it frequently changes body orientation and joint displacements due to the inaccuracy of the sensor. However, it can be suitable for other sports such as boxing, baseball, or football where the players body does not angle much in motion.

VR technology used in sports training and rehabilitation is still a research topic that needs to be understood further to produce a commercially functioning simulation. [4] As the technology advances, it will be available for average consumers to experiment which could widen the research area and development relating to sports usage.

The project was successful in terms of setting up the ground for future VR development and acquiring skills needed to produce advanced VR simulation involving physics however, as most of the time was spent learning to utilise the software with Kinect, the time spent in the actual

simulation development was short. This was an underestimation at the initial design stage and was realised halfway through the project. In continuing this project after the time frame, a clear objective of what to achieve first in making and developing based on the previous step should be considered thoroughly for the future success of this project.

Overall, I have gained valuable skills in using Unity and coding which could be used to advance the ground produced to make a VR simulation. The project was challenging but enjoyable at the same time, time management could be considered further, accounting for the unforeseen errors and issues that may arise. In all, the data acquired from the project could be useful to anyone who wants to start making a simulation on Unity.

11 Appendices

```
using UnityEngine;
using UnityEngine.XR.Interaction.Toolkit;

[RequireComponent(typeof(ActionBasedController))]
Unity Script | 0 references
public class Controller : MonoBehaviour
{
    ActionBasedController controller;

    public Hand hand;

    Unity Message | 0 references
    void Start()
    {
        controller = GetComponent<ActionBasedController>();
    }

    Unity Message | 0 references
    void Update()
    {
        //hand.SetGrip(controller.selectAction.action.ReadValue<float>());
        //hand.SetTrigger(controller.activateAction.action.ReadValue<float>());
    }
}
```

Figure 1

```
Assembly-CSharp                                   Head
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

Unity Script (1 asset reference) | 0 references
public class Head : MonoBehaviour
{
    [SerializeField] private Transform rootObject, followObject;
    [SerializeField] private Vector3 positionOffset, rotationOffset, headBodyOffset;

    Unity Message | 0 references
    private void LateUpdate()
    {
        rootObject.position = transform.position + headBodyOffset;
        rootObject.forward = Vector3.ProjectOnPlane(vector:followObject.up, planeNormal:Vector3.up).normalized;

        transform.position = followObject.TransformPoint(positionOffset);
        transform.rotation = followObject.rotation * Quaternion.Euler(rotationOffset);
    }
}
```

Figure 2

```
ead.cs   PlayerController.cs   MenuManager.cs   Panel.cs   Hand.cs   Controller.cs
Assembly-CSharp                                          PlayerController
using Unity.XR.CoreUtils;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.XR.Interaction.Toolkit;


[RequireComponent(typeof(Rigidbody))]
Unity Script (2 asset references) | 0 references
public class PlayerController : MonoBehaviour
{
    [SerializeField] private InputActionReference jumpActionReference;
    [SerializeField] private float jumpForce = 500.0f;

    private XROrigin _xrorigin;
    private CapsuleCollider _collider;
    private Rigidbody _body;

    1 reference
    private bool IsGrounded => Physics.Raycast(
        origin: (Vector3)new Vector2(x: transform.position.x, y: transform.position.y + 2.0f),
        direction: Vector3.down, maxDistance: 2.0f);

    [System.Obsolete]
    Unity Message | 0 references
    void Start()
    {
        _xrorigin = GetComponent<XROrigin>();
        _collider = GetComponent<CapsuleCollider>();
        _body = GetComponent<Rigidbody>();
        jumpActionReference.action.performed += OnJump;
    }


    Unity Message | 0 references
    void Update()
```

Figure 3

```
    Unity Message | 0 references
    void Update()
    {
        var center = _xrorigin.CameraInOriginSpacePos;
        _collider.center = new Vector3(center.x, _collider.center.y, center.z);
        _collider.height = _xrorigin.CameraInOriginSpaceHeight;
    }
        1 reference
        private void OnJump(InputAction.CallbackContext obj)
        {
            if (!IsGrounded) return;
            _body.AddForce(Vector3.up * jumpForce);
        }
    }
```

Figure 4

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.XR.Interaction.Toolkit;
Unity Script (1 asset reference) | 2 references
public class MenuManager : MonoBehaviour
{
    public Panel currentPanel = null;

        private List<Panel> panelHistory = new List<Panel>();
    Unity Message | 0 references
    private void Start()
    {

    }

    0 references
    private void SetUpPanels()
    {
        Panel[] panels = GetComponentsInChildren<Panel>();

        foreach(Panel panel in panels)
            panel.Setup(this);

        currentPanel.Show();
    }

    Unity Message | 0 references
    private void Update()
    {
        //if(OVRInput.GetDown(OVRInput.Button.PrimaryHandTrigger))
            GoToPrevious();
    }
    1 reference
    public void GoToPrevious()
```

Figure 5

```
    Unity Message | 0 references
    private void Update()
    {
        //if(OVRInput.GetDown(OVRInput.Button.PrimaryHandTrigger))
            GoToPrevious();
    }
    1 reference
    public void GoToPrevious()
    {
        if (panelHistory.Count == 0)
            return;
        int lastIndex = panelHistory.Count - 1;
        SetCurrent(panelHistory[lastIndex]);
        panelHistory.RemoveAt(lastIndex);
    }

    0 references
    public void SetCurrentWithHistory(Panel newPanel)
    {
        panelHistory.Add(currentPanel);
        SetCurrent(newPanel);
    }

    2 references
    private void SetCurrent(Panel newPanel)
    {
        currentPanel.Hide();

        currentPanel = newPanel;
        currentPanel.Show();

    }
}
```

Figure 6

```
using UnityEngine;

Unity Script (2 asset references) | 8 references
public class Panel : MonoBehaviour
{
    private Canvas canvas = null;
    private MenuManager menuManager = null;

    Unity Message | 0 references
    private void Awake()
    {
        canvas = GetComponent<Canvas>();
    }

    1 reference
    public void Setup(MenuManager menuManager)
    {
        this.menuManager = menuManager;
        Hide();
    }

    2 references
    public void Show()
    {
        canvas.enabled = true;
    }

    2 references
    public void Hide()
    {
        canvas.enabled = false;
    }
}
```

Figure 7

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

[RequireComponent(typeof(Animator))]

Unity Script (6 asset references) | 1 reference
public class Hand : MonoBehaviour
{
    // Animation
    [SerializeField] private float animiationSpeed;
    SkinnedMeshRenderer mesh;
    Animator animator;
    private float gripTarget;
    private float triggerTarget;
    private float gripCurrent;
    private float triggerCurrent;
    private const string animatorGripParam = "Grip";
    private const string animatorTriggerParam = "Trigger";
    private static readonly int Grip = Animator.StringToHash(name: animatorGripParam);
    private static readonly int Trigger = Animator.StringToHash(name: animatorTriggerParam);

    //Physics Movement
    [SerializeField] private GameObject followObject;
    [SerializeField] private float followSpeed = 30f;
    [SerializeField] private float rotateSpeed = 100f;
    [SerializeField] private Vector3 positionOffset;
    [SerializeField] private Vector3 rotationOffset;
    private Transform _followTarget;
    private Rigidbody _body;
    // Start is called before the first frame update
    Unity Message | 0 references
```

Figure 8

```
    void Start()
    {
        //Animation
        animator = GetComponent<Animator>();
        mesh = GetComponentInChildren<SkinnedMeshRenderer>();

        //Physics Movement
        _followTarget = followObject.transform;
        _body = GetComponent<Rigidbody>();
        _body.collisionDetectionMode = CollisionDetectionMode.Continuous;
        _body.interpolation = RigidbodyInterpolation.Interpolate;
        _body.mass = 20f;

        //Teleport hands
        _body.position = _followTarget.position;
        _body.rotation = _followTarget.rotation;

    }
```

Figure 9

```
52  void Update()
53  {
54      AnimateHand();
55
56      PhysicsMove();
57
58  }
59
    1 reference
60  private void PhysicsMove()
61  {
62      //Position
63      var positionWithOffset = _followTarget.TransformPoint(positionOffset);
64
65      var distance = Vector3.Distance(a: positionWithOffset, b:transform.position);
66      _body.velocity = (positionWithOffset - transform.position).normalized * (followSpeed * distance);
67      //Rotation
68      var rotationWithOffset = _followTarget.rotation * Quaternion.Euler(rotationOffset);
69      var q = rotationWithOffset * Quaternion.Inverse(_body.rotation);
70      q.ToAngleAxis(out float angle, out Vector3 axis);
71      _body.angularVelocity = axis * (angle * Mathf.Deg2Rad * rotateSpeed);
72  }
73
```

Figure 10

```
74  internal void SetGrip(float v)
75  {
76      gripTarget = v;
77  }
78
    0 references
79  internal void SetTrigger(float v)
80  {
81      triggerTarget = v;
82  }
83
    1 reference
84  void AnimateHand()
85  {
86      if (gripCurrent != gripTarget)
87      {
88          gripCurrent = Mathf.MoveTowards(gripCurrent, gripTarget, Time.deltaTime * animiationSpeed);
89          animator.SetFloat(animatorGripParam, gripCurrent);
90      }
91      if (triggerCurrent != triggerTarget)
92      {
93          triggerCurrent = Mathf.MoveTowards(triggerCurrent, triggerTarget, Time.deltaTime * animiationSpeed);
94          animator.SetFloat(animatorTriggerParam, triggerCurrent);
95      }
96  }
```

Figure 11

```
0 references
internal void SetTrigger(float v)
{
    triggerTarget = v;
}

1 reference
void AnimateHand()
{
    if (gripCurrent != gripTarget)
    {
        gripCurrent = Mathf.MoveTowards(gripCurrent, gripTarget, Time.deltaTime * animiationSpeed);
        animator.SetFloat(animatorGripParam, gripCurrent);
    }
    if (triggerCurrent != triggerTarget)
    {
        triggerCurrent = Mathf.MoveTowards(triggerCurrent, triggerTarget, Time.deltaTime * animiationSpeed);
        animator.SetFloat(animatorTriggerParam, triggerCurrent);
    }
}

0 references
public void ToggleVisibility()
{
    mesh.enabled = !mesh.enabled;
}

}
```

Figure 12

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
Unity Script | 0 references
public class ImageViewer : MonoBehaviour
{
    public MultiSourceManager _multiSource;
    public MeasureDepth _MeasureDepth;
    public RawImage _RawImage;
    public RawImage _RawDepth;
    Unity Message | 0 references
    void Update()
    {
        _RawImage.texture = _multiSource.GetColorTexture();

        _RawDepth.texture = _MeasureDepth._DepthTexture;
    }
}
```

Figure 13

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using Windows.Kinect;

public class MeasureDepth : MonoBehaviour
{
    public MultiSourceManager _MultiSource;
    public Texture2D _DepthTexture;

    private ushort[] _DepthData;
    private CameraSpacePoint[] _CameraSpacePoints = null;
    private ColorSpacePoint[] _ColorSpacePoints = null;

    private KinectSensor _Sensor = null;
    private CoordinateMapper _Mapper = null;

    private readonly Vector2Int _DepthResolution = new Vector2Int(512, 424);

```

Figure 14

```
Unity Message | 0 references
private void Awake()
{
    _Sensor = KinectSensor.GetDefault();
    _Mapper = _Sensor.CoordinateMapper;

    int arraySize = _DepthResolution.x * _DepthResolution.y;

    _CameraSpacePoints = new CameraSpacePoint[arraySize];
    _ColorSpacePoints = new ColorSpacePoint[arraySize];
}

Unity Message | 0 references
private void Update()
{
   // if (Input.GetKeyDown(KeyCode.Space))
    {
        DepthToColor();

        _DepthTexture = CreateTexture();
    }
}
1 reference
private void DepthToColor()
{//Get Depth
    _DepthData = _MultiSource.GetDepthData();

    //Map
    _Mapper.MapDepthFrameToCameraSpace(_DepthData, _CameraSpacePoints);
    _Mapper.MapDepthFrameToColorSpace(_DepthData, _ColorSpacePoints);
}
```

Figure 15

```
1 reference
private Texture2D CreateTexture()
{
    Texture2D newTexture = new Texture2D(1920, 1080, TextureFormat.Alpha8, false);

    for(int x = 0; x < 1920; x++)
    {
        for(int y = 0; y < 1080; y++)
        {
            newTexture.SetPixel(x, y, Color.clear);
        }
    }

    foreach(ColorSpacePoint point in _ColorSpacePoints)
    {
        newTexture.SetPixel((int)point.X, (int)point.Y, Color.black);
    }

    newTexture.Apply();

    return newTexture;
}
}
```

Figure 16

12 References

1. Shingade A, Ghotkar A. Animation of 3D Human Model Using Markerless Motion Capture Applied To Sports [Internet]. Arxiv.org. 2014 [cited 15 February 2022]. Available from: https://arxiv.org/ftp/arxiv/papers/1402/1402.2363.pdf

2. Groh B, Flaschka J, Wirth M, Kautz T, Fleckenstein M, Eskofier B. Wearable Real-Time Skateboard Trick Visualization and Its Community Perception [Internet]. Ieeexplore.ieee.org. 2016 [cited 20 February 2022]. Available from: https://ieeexplore.ieee.org/abstract/document/7579423

3. Chan J, Leung H, Tang J, Komura T. A Virtual Reality Dance Training System Using Motion Capture Technology [Internet]. Ieeexplore.ieee.org. 2011 [cited 24 February 2022]. Available from: https://ieeexplore.ieee.org/abstract/document/5557840/references#references

4. Lopez Rincon A, Yamasaki H, Shimoda S. Design of a video game for rehabilitation using motion capture, EMG analysis and virtual reality [Internet]. Ieeexplore.ieee.org. 2016 [cited 3 March 2022]. Available from: https://ieeexplore.ieee.org/abstract/document/7438575

5. Napoli A, Glass S, Ward C, Tucker C, Obeid I. Performance analysis of a generalized motion capture system using microsoft kinect 2.0 [Internet]. ScienceDirect. 2017 [cited 3 March 2022]. Available from: https://www.sciencedirect.com/science/article/abs/pii/S174680941730112X

6. Download Kinect for Windows SDK 2.0 from Official Microsoft Download Center [Internet]. Microsoft.com. 2015 [cited 4 March 2022]. Available from: https://www.microsoft.com/en-us/download/details.aspx?id=44561

7. Asteriadis S, Chatzitofis A, Zarpalas D, Alexiadis D, Daras P. Estimating human motion from multiple Kinect sensors | Proceedings of the 6th International Conference on Computer Vision / Computer Graphics Collaboration Techniques and Applications [Internet]. ACM Other conferences. 2013 [cited 12 March 2022]. Available from: https://dl.acm.org/doi/abs/10.1145/2466715.2466727

8. Alabbasi H, Gradinaru A, Moldoveanu F, Moldoveanu A. Human motion tracking & evaluation using Kinect V2 sensor [Internet]. Ieeexplore.ieee.org. 2016 [cited 10 March 2022]. Available from: https://ieeexplore.ieee.org/abstract/document/7391465

9. Tian Y, Meng X, Taoc D, Liu D, Feng C. Upper limb motion tracking with the integration of IMU and Kinect [Internet]. ScienceDirect. 2015 [cited 11 March 2022]. Available from: https://www.sciencedirect.com/science/article/abs/pii/S0925231215001472

10. XR Interaction Toolkit | XR Interaction Toolkit | 0.9.4-preview [Internet]. Docs.unity3d.com. 2022 [cited 12 March 2022]. Available from: https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@0.9/manual/index.html

11. Hoxey T, Stephenson I. Integrating Live Skeleton data into a VR Environment [Internet]. Eprints.bournemouth.ac.uk. 2018 [cited 14 March 2022]. Available from: http://eprints.bournemouth.ac.uk/30826/1/integrating-live-skeleton.pdf

12. STRIVE - Science [Internet]. STRIVE. 2022 [cited 2 February 2022]. Available from: https://strive.tech/science/

13. NBA League Averages - Per Game | Basketball-Reference.com [Internet]. Basketball-Reference.com. 2022 [cited 10 March 2022]. Available from: https://www.basketball-reference.com/leagues/NBA_stats_per_game.html

14. Li D, Yi C, Gu Y. Research on College Physical Education and Sports Training Based on Virtual Reality Technology [Internet]. Hindawi. 2021 [cited 20 March 2022]. Available from: https://www.hindawi.com/journals/mpe/2021/6625529/